

Presentation Script - NYC Taxi Trip Duration (Stages 1-5)

Introduction

What was expected of me: to define the business problem clearly, generate testable hypotheses about what might affect trip duration, collect and inspect the raw dataset, explore it thoroughly, then take the cleaned EDA dataset and prepare it properly for machine learning - checking data quality one final time, handling outliers with justified reasoning, engineering a useful feature, splitting and scaling the data correctly, training a KNN model and a Linear Regression model, tuning and comparing them, and diagnosing the results using residual analysis.

STAGE 1 - PROBLEM DEFINITION

Step 1: Defining the Problem (TOSCAR Method)

Purpose: Before touching any data, the problem itself needs to be clearly defined. I used the TOSCAR method - a checklist of 6 questions (Trouble, Owner, Success Criteria, Constraints, Actors, References) - to make sure the problem was properly scoped before jumping into the data.

Report: "I used the TOSCAR method to properly scope the problem before touching any data. The trouble is that riders and dispatchers only receive accurate trip-duration information after a trip has already ended, so ETAs given beforehand are unreliable. The owner is the taxi companies and dispatchers, since they are directly responsible for allocating the fleet. Success would mean more accurate duration estimates before a trip begins, fewer rider complaints, and better fleet allocation - although since this notebook only covers Problem Definition through EDA, there is no model yet to actually evaluate against that criteria. The main constraint is that only historical trip records are available, with no live traffic or weather feed, and any future prediction would need to run fast enough for near-real-time dispatch. The actors who interact with the solution are passengers, drivers, dispatchers/operations managers, and the taxi vendors themselves. The reference data available is the historical NYC TLC Yellow Cab dataset, containing pickup/dropoff coordinates, timestamps, vendor info, passenger counts, and actual trip durations."

Final Problem Statement: "Trip duration for NYC taxi rides is unpredictable, causing dispatchers to misallocate taxis, passengers to receive unreliable ETAs, drivers to lose income from inefficient scheduling, and taxi companies to lose customer trust and revenue."

Guiding question: "Which factors in a taxi trip's pickup information can help us understand and estimate trip duration before the trip begins?"

Why TOSCAR specifically: it forces you to justify the problem from multiple angles (who is affected, what limits the solution, what data actually exists) rather than jumping straight to "let's predict trip duration" without understanding why it matters or who it's for.

STAGE 2 - HYPOTHESIS GENERATION

Step 2: Generating Hypotheses

Purpose: Before exploring the data, I wrote down what I expected to find, based on general knowledge of how taxi trips work. This gives the EDA a clear direction - each visualisation and observation report can then be checked against a hypothesis, instead of just describing charts with no purpose.

H1 : Trips covering a longer distance take longer.
H2 : Trips during rush hour (7 to 9am, 4 to 7pm) take longer than off-peak trips.
H3 : Weekday trips take longer than weekend trips.
H4 : Trips with more passengers may take slightly longer (extra stops/loading).
H5 : Vendor (vendor_id 1 vs 2) may show different average durations.
H6 : store_and_fwd_flag = Y trips (server connectivity lost) may behave differently - a data quality flag, not necessarily a real driver of duration.

Report: "I generated six hypotheses before exploring the data, covering distance, time of day, day of the week, passenger count, vendor, and the store-and-forward flag. Each one is written so it can actually be tested against the data later - for example H2 specifies exact rush-hour windows (7-9am, 4-7pm) rather than just saying 'time of day matters', so the EDA can confirm or refine it. H6 is treated a bit differently from the others - it's flagged upfront as more likely to be a data-quality flag than a genuine driver of duration, since a lost server connection has no real reason to make a trip itself take longer."

STAGE 3 - DATA EXTRACTION / COLLECTION

Step 3: Importing Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
```

Explanation: These are the core Python libraries used throughout the notebook. pandas handles the data table (loading, filtering, grouping); numpy handles numerical operations; matplotlib and seaborn are used for all the charts; scipy.stats is available for any statistical checks needed later.

Step 4: Loading the Dataset

```
data = pd.read_csv('NYC.csv')
```

Purpose of this code: Loads the raw NYC Taxi Trip Duration dataset from a CSV file into a pandas DataFrame called data, so it can be inspected and worked with in the rest of the notebook.

Step 5: File Structure and Data Types

data.head() / data.tail() - shows the first and last 5 rows, as a quick visual check of the data (not for analysis), confirming the columns look as expected: id, vendor_id, pickup/dropoff datetime, passenger_count, pickup/dropoff coordinates, store_and_fwd_flag, and trip_duration.

```
data.shape
# Output: (1458644, 11)
```

Report: The dataset contains 1,458,644 trips and 11 columns.

```
data.dtypes
# Output:
id str
vendor_id int64
pickup_datetime str
dropoff_datetime str
passenger_count int64
pickup_longitude float64
pickup_latitude float64
dropoff_longitude float64
```

```
dropoff_latitude float64
store_and_fwd_flag str
trip_duration int64
```

Report: "I used data.dtypes to check the raw data type of every column straight after loading the dataset. At this stage pickup_datetime and dropoff_datetime are still plain text (str), and store_and_fwd_flag is also text ('Y'/'N') rather than numeric - both of these get converted properly in the EDA stage before they can be used for analysis or plotting."

That completes Stage 3 - the raw dataset has been loaded and its structure confirmed, ready for exploration in Stage 4.

STAGE 4 - DATA EXPLORATION AND TRANSFORMATION (EDA)

Step 6: Missing Values Check

```
data.isnull().sum()
# Output: 0 for every column
```

Report: No missing values were found in any column of the raw dataset. Therefore no imputation was needed at this stage.

Step 7: Reformatting Features

Purpose: Two columns needed to be converted before they could be used properly: the pickup/dropoff timestamps (currently plain text) needed to become real datetime objects so date/time parts could be extracted from them, and the store_and_fwd_flag (currently 'Y'/'N' text) needed to become numeric so it could eventually be used by a model.

```
# Part 1: Converting strings to datetime features
data['pickup_datetime'] = pd.to_datetime(data['pickup_datetime'])
data['dropoff_datetime'] = pd.to_datetime(data['dropoff_datetime'])

# Part 2: Converting the yes/no flag to 1 and 0
data['store_and_fwd_flag'] = data['store_and_fwd_flag'].astype('category')
data['store_and_fwd_flag'] = data['store_and_fwd_flag'].cat.codes
```

Code explanation: pd.to_datetime() converts the text timestamps into proper datetime64 objects, which allows date/time parts (weekday, hour) to be pulled out later using .dt accessors. Converting store_and_fwd_flag to a category and then to .cat.codes turns the text values ('N'/'Y') into numeric codes (0/1), so the column can be used numerically later - N became 0, Y became 1.

```
data['store_and_fwd_flag'].value_counts()
# Output:
0 1450599
1 8045
```

Report: After conversion, 1,450,599 trips have store_and_fwd_flag = 0 (no connectivity issue) and 8,045 trips have store_and_fwd_flag = 1 (data was stored before being forwarded).

Step 8: Target Exploration (trip_duration statistics)

```
data['trip_duration'].describe()
# Output:
count 1.458644e+06
mean 9.594923e+02
```

```
std 5.237432e+03
min 1.000000e+00
25% 3.970000e+02
50% 6.620000e+02
75% 1.075000e+03
max 3.526282e+06
```

Report: "There are 1,458,644 trips with no missing values. The average trip is about 960 seconds (16 minutes), but the standard deviation (5,237 seconds) is over 5 times bigger than the mean itself - that's a major red flag telling you the data has extreme outliers dragging the average around. The median (662 seconds, about 11 minutes) is a much more realistic 'typical trip' than the mean. The maximum value (3,526,282 seconds, about 40.8 days) confirms this - no real taxi trip lasts 40 days, so this is clearly a data error or extreme outlier that needs to be handled. Since mean (959.5) is greater than median (662), the distribution is right-skewed, meaning there's a long tail of unusually high values pulling the average upward."

Step 9: Log Transform of trip_duration

```
data['trip_duration'].hist(bins=50)
plt.title('Distribution of trip_duration BEFORE log transform')
plt.show()

data['log_trip_duration'] = np.log1p(data['trip_duration'])
data['log_trip_duration'].hist(bins=50)
plt.title('Distribution of trip_duration AFTER log transform')
plt.savefig('trip_duration_after_log.pdf')
plt.show()
```

Purpose of this code: The raw trip_duration histogram is extremely right-skewed (confirmed in Step 8), which makes it hard to visually see any pattern - almost every trip gets squashed into one tall bar near zero, with the outliers stretching the x-axis out to almost nothing useful. Applying a log transform (np.log1p, which is $\log(1 + x)$, so it can handle any trips at exactly 0 safely) compresses the very large values and spreads out the smaller ones, giving a much more readable, closer to bell-shaped distribution to look at. This log_trip_duration column is used only for visualising the shape of the distribution here - it is not used as the model's actual target, and is dropped later during preprocessing to avoid target leakage, since it's directly derived from trip_duration.

Report: The before/after histograms show the log transform successfully turns the heavily right-skewed raw distribution into a much more symmetrical, bell-shaped one, confirming visually what the describe() statistics in Step 8 already suggested about the skew.

Step 10: Univariate - Passenger Count

```
data['passenger_count'].value_counts().sort_index()
data['passenger_count'].value_counts(normalize=True) * 100
data['passenger_count'].value_counts().sort_index().plot(kind='bar')
```

Observation Report: "Most taxi trips carry just 1 passenger with 1,033,540 trips, well over 70% of the entire dataset. This is by far the dominant pattern, confirming NYC taxis are mostly used for solo rides. 2 passengers is the next most common (210,318 trips), and numbers drop off steadily from there for 3, 4, 5, and 6 passengers. On the outliers: 0 passengers (60 trips) is a confirmed outlier and data error - a taxi cannot legally dispatch with zero passengers. 5 and 6 passengers (78,088 and 48,333 trips) are a genuine grey area, not confirmed either way, since standard sedans are typically licensed for only 4 passengers, so these could be larger accessible vehicles or could be errors. 7, 8, 9 passengers (1-3 trips each) are too rare to represent any real pattern and are almost certainly errors or extreme edge cases."

Step 11: Univariate - Vendor ID

```
data['vendor_id'].value_counts()  
data['vendor_id'].value_counts(normalize=True) * 100  
data['vendor_id'].value_counts().plot(kind='bar')
```

Observation Report: Vendor 1 recorded 678,342 trips (46.5% of the dataset), while Vendor 2 recorded 780,302 trips (53.5%). The split between the two vendors is fairly balanced, with Vendor 2 handling a slightly larger share of total trips - roughly 100,000 more than Vendor 1, but not an extreme imbalance either way.

Step 12: Univariate - Store and Forward Flag

```
data['store_and_fwd_flag'].value_counts()  
data['store_and_fwd_flag'].value_counts(normalize=True) * 100  
data['store_and_fwd_flag'].value_counts().plot(kind='bar')
```

Observation Report: "Most trips - 1,450,599 (99.45%) - have store_and_fwd_flag = 0, meaning trip data was sent to the server right away with no connectivity issues. Only 8,045 trips (0.55%) have store_and_fwd_flag = 1. Regarding H6: this analysis only examines the frequency of store_and_fwd_flag values, it does not compare trip_duration between the two groups. Because the flag = 1 group is extremely small (only 0.55% of trips), any comparison would need to be interpreted cautiously due to the small sample size. H6 is therefore marked as not tested, rather than supported or rejected, since the relationship with trip duration was not directly analysed."

Step 13: Pickup Time - Weekday and Hour

```
data['pickup_weekday'] = data['pickup_datetime'].dt.day_name()  
data['pickup_hour'] = data['pickup_datetime'].dt.hour  
  
weekday_order = ['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday', 'Sunday']  
data['pickup_weekday'].value_counts().reindex(weekday_order).plot(kind='bar')  
  
data['pickup_hour'].value_counts().sort_index().plot(kind='bar')
```

Code explanation: dt.day_name() extracts the weekday name (e.g. 'Monday') from the full pickup timestamp into a new column, and dt.hour extracts just the hour (0-23). These become the day_of_week and hour_of_day features used throughout the rest of the analysis and later in modelling.

Observation Report: "Weekday pattern: Friday has the most pickups (223,533), followed closely by Saturday (220,868) and Thursday (218,574). Monday has the fewest (187,418). This shows a clear weekly pattern - pickups build up through the week, peak around Thursday/Friday, stay high through Saturday, then drop off on Sunday and especially Monday. This supports H3, but with one twist: it's not simply 'weekdays busier than weekends' - Friday and Saturday, which are really the start of the weekend, are actually the busiest days, while Sunday and Monday are the quietest. Hourly pattern: pickups are lowest in the early morning hours (around 2am-5am), then steadily rise through the day, reaching their highest volume in the evening. Hour 0 (midnight to 12:59am) alone still has about 53,000 trips, showing real demand even at the quietest part of the day, likely from nightlife and late travel. Both patterns directly support H2 (rush hour) and H3 (weekday vs weekend): there's a real, measurable difference in when people take taxis, which sets up the expectation that trip duration should also vary by time, since more demand at certain hours/days usually means more traffic too. This gets tested properly in the bivariate section next."

Why pickup_hour is useful

Demand patterns: rush-hour peaks (8-9am, 6-7pm) versus low demand (2-5am). Traffic conditions: trips during peak hours often take longer due to congestion. Modelling impact: including pickup_hour helps the model adjust predictions for trip duration depending on time of day.

"Pickup_hour is an important feature because trip demand and traffic vary by time of day. While trip_distance is the strongest predictor, pickup_hour adds explanatory power by capturing rush-hour effects."

"This bar chart shows pickups by hour of day. Demand is lowest around 3-5am, rises during the morning commute, stays steady midday, and peaks at 6-8pm. This confirms that pickup time is a strong factor in taxi demand and trip duration, since traffic and demand vary systematically by hour."

Step 14: Pickup/Dropoff Coordinate Distributions

Purpose: Checking whether the location data (pickup/dropoff coordinates) is clean, realistic, and consistent, before trusting it for any later analysis or mapping.

```
data[['pickup_latitude', 'pickup_longitude',  
      'dropoff_latitude', 'dropoff_longitude']].describe()
```

What .describe() does

It generates a statistical summary of the selected numeric columns. For each column (pickup_latitude, pickup_longitude, dropoff_latitude, dropoff_longitude), you get: count (number of non-missing values), mean (average value), std (standard deviation / spread), min (smallest value), 25% / 50% / 75% (quartiles - distribution checkpoints), and max (largest value).

count: "The count row shows that all four coordinate columns have 1,458,644 entries. This means every trip has both pickup and dropoff coordinates recorded, with no missing values. That makes the dataset complete and ready for analysis."

mean: The mean is the average of all values in each column. Since pickup and dropoff means are almost identical, it shows that trips generally start and end around the same central area in the dataset. Yes, 40.75 and -73.97 are coordinates - they are the average pickup and dropoff positions in the dataset. Together, they represent the central point of the dataset.

std: "The standard deviation numbers are small, which means most coordinates don't move far from the average. Trips are generally in the same area, and only a few values are spread out more widely." std (approximately 0.03-0.07) is a very small spread - most coordinates are tightly grouped near the mean.

min: "The minimum row shows the lowest recorded coordinates. Since these are far away from the average and quartile values, they don't fit the usual pattern of the data. That's why they are treated as outliers."

25% / 50% / 75% (quartiles): "The quartile rows show that most trips happen around the same area. The middle half of trips (between 25% and 75%) are very close together, and pickup and dropoff values match closely. This proves the dataset is consistent and trips usually occur in the same region."

Quartile evidence: 25% (Q1) pickup latitude = 40.737, longitude = -73.9919. 50% (median) pickup latitude = 40.7541, longitude = -73.9817. 75% (Q3) pickup latitude = 40.768, longitude = -73.9673. The differences are small: latitude changes only about 0.03 between Q1 and Q3, and longitude changes only about 0.02 between Q1 and Q3.

max: "The maximum row shows the highest recorded coordinates. They are much higher than the typical values (quartiles and median), which means they don't fit the usual pattern of trips. These are extreme values, likely outliers."

Most of the numbers (the mean and the quartiles) sit very close together - that shows the "normal range" of values. The minimum and maximum are much further away from that normal range. Because they don't fit with the rest of the values, we call them outliers.

"Min and max are just the smallest and largest values. We call them outliers only when they sit far outside the usual range of the data. The usual range is shown by the quartiles and standard deviation. If min and max are way beyond that, they don't match the pattern - that's why they're considered outliers."

IQR rule (formal test)

Outliers are values less than Q1 minus 1.5xIQR, or greater than Q3 plus 1.5xIQR. This is the statistical way to prove min/max are outliers: $IQR = Q3 - Q1$; Lower bound = $Q1 - (1.5 \times IQR)$; Upper bound = $Q3 + (1.5 \times IQR)$. Any value outside these bounds is an outlier.

Pickup Latitude: $Q1 = 40.737$, $Q3 = 40.768$, so $IQR = 0.031$. Lower bound = $40.737 - (1.5 \times 0.031) = 40.691$. Upper bound = $40.768 + (1.5 \times 0.031) = 40.814$. Min = 34.36, Max = 51.88 - both outside the bounds, so both are outliers. Normal values should fall between 40.691 and 40.814; the min (34.36) is way below 40.691, and the max (51.88) is way above 40.814.

Pickup Longitude: $Q1 = -73.9919$, $Q3 = -73.9673$, so $IQR = 0.0246$. Lower bound = $-73.9919 - (1.5 \times 0.0246) = -74.0298$. Upper bound = $-73.9673 + (1.5 \times 0.0246) = -73.9294$. Min = -121.93, Max = -61.33 - both outside the bounds, both outliers.

Dropoff Latitude: $Q1 = 40.736$, $Q3 = 40.7698$, so $IQR = 0.0338$. Lower bound = $40.736 - (1.5 \times 0.0338) = 40.685$. Upper bound = $40.7698 + (1.5 \times 0.0338) = 40.821$. Min = 32.18, Max = 43.92 - both outside the bounds, both outliers.

Dropoff Longitude: $Q1 = -73.9913$, $Q3 = -73.9630$, so $IQR = 0.0283$. Lower bound = $-73.9913 - (1.5 \times 0.0283) = -74.0347$. Upper bound = $-73.9630 + (1.5 \times 0.0283) = -73.9196$. Min = -121.93, Max = -61.33 - both outside the bounds, both outliers.

```
# checking if a value falls inside that range (returns True or False for every row)
nyc_box = data[(data['pickup_longitude'].between(-74.05, -73.75)) &
               (data['pickup_latitude'].between(40.6, 40.9)) &
               (data['dropoff_longitude'].between(-74.05, -73.75)) &
               (data['dropoff_latitude'].between(40.6, 40.9))]
```

"This code keeps only the trips where both pickup and dropoff coordinates fall inside the NYC map range. It's a way to clean the dataset by removing trips with impossible or far-away coordinates." The boundary values are the actual latitude and longitude limits of New York City, chosen to filter out trips that fall outside the city. Anything outside that box is not a real NYC taxi trip, so the filter removes it.

Evidence from the data: Pickup latitude $Q1 = 40.737$, $Q3 = 40.768$ (normal NYC range), Min = 34.36, Max = 51.88 (far outside). Pickup longitude $Q1 = -73.9919$, $Q3 = -73.9673$ (normal NYC range), Min = -121.93, Max = -61.33 (far outside). Dropoff latitude $Q1 = 40.736$, $Q3 = 40.7698$ (normal NYC range), Min = 32.18, Max = 43.92 (far outside). Dropoff longitude $Q1 = -73.9913$, $Q3 = -73.9630$ (normal NYC range), Min = -121.93, Max = -61.33 (far outside).

"I know there were trips outside NYC because the minimum and maximum coordinate values are far away from the normal quartile range. The IQR rule shows they don't fit the usual NYC pattern, so they are outliers and must be removed."

```
fig, axes = plt.subplots(2, 2, figsize=(12,8))

cols = ['pickup_longitude', 'pickup_latitude', 'dropoff_longitude', 'dropoff_latitude']
titles = ['Pickup Longitude', 'Pickup Latitude', 'Dropoff Longitude', 'Dropoff Latitude']
positions = [(0,0), (0,1), (1,0), (1,1)]

for col, title, pos in zip(cols, titles, positions):
```

```

ax = axes[pos]
sns.kdeplot(nyc_box[col], ax=ax, fill=True)

mean_val = nyc_box[col].mean()
median_val = nyc_box[col].median()

ax.scatter([mean_val], [0], color='red', marker='o', s=60, label='mean', zorder=5)
ax.scatter([median_val], [0], color='blue', marker='o', s=60, label='median', zorder=5)

ax.set_title(title)
ax.legend()

plt.tight_layout()
plt.show()

```

Here I am plotting distribution charts (KDE plots) for the four coordinate columns: pickup longitude, pickup latitude, dropoff longitude, dropoff latitude. Each chart shows where most values are clustered and highlights the mean (red dot) and median (blue dot). Purpose: to check if trips fall in the expected NYC range and to spot unusual values (outliers).

Pickup Longitude: the curve shows most pickup points around -73.9. Mean and median are close together, meaning the data is balanced. If there were extreme values, the curve would stretch.

Pickup Latitude: most pickups are around 40.7. Again, mean is approximately equal to median, so no big skew. This confirms pickups are in the NYC area.

Dropoff Longitude: similar to pickup longitude, clustered around -73.9. Mean and median overlap, showing consistent trips.

Dropoff Latitude: cluster around 40.7. Mean and median are close, showing trips end in the same region as pickups.

Observation Report: "All four coordinates cluster very tightly together: mean and median sit within 0.01 of each other for every single column, meaning these distributions are close to symmetrical, unlike the heavily right-skewed trip_duration. This makes sense geographically - the coordinates cluster around a single small area consistent with central New York City, confirmed visually against real geography using the folium map later. Before filtering, the raw min/max values (latitude up to 51.88, longitude down to -121.93) confirm a small number of GPS-logging errors - coordinates that place a 'New York taxi trip' outside the actual United States entirely. These rows were excluded from the chart to keep it readable, though they remain in the main dataset unless explicitly removed later for modelling."

Step 15: Bivariate - Trip Duration vs Day of Week and Hour

Bivariate Relations with Target - after exploring the individual data, plot trip duration against day of the week, and trip duration against hour of day. We are checking how the target variable (trip_duration) relates to two categorical/time features: day of the week (Monday-Sunday) and hour of day (0-23). In bivariate analysis, two variables are compared at a time: trip duration vs weekday, and trip duration vs hour.

```

weekday_order = ['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday', 'Sunday']
data.groupby('pickup_weekday')['trip_duration'].mean().reindex(weekday_order)

```

What the code does: trips are grouped by pickup_weekday, then the mean trip_duration is calculated for each day, and finally reindexed with weekday_order so the days appear in natural order (Mon to Sun).

```
pickup_weekday
Monday 897.947839
Tuesday 983.463124
Wednesday 975.450494
Thursday 1006.528654
Friday 990.224200
Saturday 948.051175
Sunday 901.639395
```

How to explain the results: Monday, about 898 seconds (~15 min), shortest average trips. Tuesday, about 983 seconds (~16 min), trips slightly longer. Wednesday, about 975 seconds (~16 min), similar to Tuesday. Thursday, about 1,007 seconds (~16.7 min), longest average trips. Friday, about 990 seconds (~16.5 min), still high. Saturday, about 948 seconds (~15.8 min), shorter again. Sunday, about 902 seconds (~15 min), close to Monday, also short.

```
plt.figure(figsize=(10,4))
sns.boxplot(x='pickup_weekday', y='trip_duration', data=data,
order=weekday_order, showfliers=False)
plt.xlabel('Day of Week')
plt.ylabel('Trip Duration (seconds)')
plt.title('Trip Duration vs Day of Week')
plt.show()
```

Each box represents the distribution of trip durations for one weekday. The line inside the box is the median trip duration, and the box edges are Q1 and Q3 (the middle 50% of trips).

How to explain the visuals: Monday and Sunday - boxes are lower, so shorter average trips, median around ~900 seconds. Tuesday, Wednesday, Friday - slightly higher boxes, so longer trips, median ~975-990 seconds. Thursday - tallest box, longest trips, median ~1,006 seconds, showing Thursday trips take more time. Saturday - slightly shorter again, closer to the weekday average.

```
data.groupby('pickup_hour')['trip_duration'].mean() # Average Trip Duration by Hour
```

This groups trips by pickup_hour (0-23), calculates the mean trip_duration for each hour, and shows how trip length changes depending on the time of day.

```
pickup_hour
0 936.657302
1 903.087112
2 890.076934
3 890.070878
4 921.976444
5 822.298960
6 797.434943
7 831.758345
8 924.559199
9 933.528915
10 933.367605
11 966.430282
12 993.533079
13 1032.246359
14 1075.789264
15 1118.832533
16 1080.146285
17 1030.581619
18 981.583234
19 894.356679
20 879.461545
21 890.521221
```

```
22 1023.455921
23 925.091409
```

How to read the results: early morning (0-5h), trip durations around 890-936 seconds (15-16 min). Morning commute (6-9h), shorter trips (797-933 seconds), likely due to short work commutes. Midday (10-15h), durations rise steadily (832 to 1,118 seconds), longer trips during the day. Evening commute (16-17h), still high (~1,030-1,080 seconds). Late evening (18-23h), mixed - some hours shorter (~890-925), but a spike again at 21h (~1,023 seconds).

```
plt.figure(figsize=(12,4))
sns.boxplot(x='pickup_hour', y='trip_duration', data=data, showfliers=False)
plt.xlabel('Hour of Day')
plt.ylabel('Trip Duration (seconds)')
plt.title('Trip Duration vs Hour of Day')
plt.show()
```

What this code does: creates a boxplot for each hour (0-23), showing the distribution of trip durations per hour (median as the line inside the box, quartiles as the box edges, range as the whiskers). `showfliers=False` hides extreme outliers so the plot focuses on the main distribution.

How to explain the visuals: early morning (0-5h) - boxes are lower, showing shorter trips. Morning commute (6-9h) - trips dip further (median ~800-930s), reflecting short work commutes. Midday (10-15h) - boxes rise steadily, showing longer trips (median peaks ~1,100s around 3pm). Evening commute (16-17h) - still high, trips remain long. Late evening (18-23h) - mixed pattern, some hours shorter (~900s), but spikes again around 21h.

Observation Report: "Day of week: average trip duration is lowest on Monday (898s) and Sunday (902s), and highest midweek, peaking on Thursday (1,007s). This supports H3 - weekday trips genuinely take longer than weekend trips, likely due to commuter traffic. Hour of day: average trip duration is lowest in the early morning (6am, 797s) and highest in the afternoon (3pm, 1,119s), with a second smaller peak around 10pm (1,024s). H2 is partially supported: trip duration does vary meaningfully by hour, but the true peak isn't the classic 7-9am or 4-7pm rush hour windows specified in the hypothesis - it's mid-afternoon (2-4pm), so the hypothesis captures the general pattern but not the specific timing."

Step 16: Vendor ID vs Trip Duration

Checking whether trip duration differs between vendors (Vendor 1 and Vendor 2). This helps test if vendor choice affects how long trips take - part of the hypothesis about pickup factors.

```
data.groupby('vendor_id')['trip_duration'].mean()
```

This code calculates the average trip duration separately for Vendor 1 and Vendor 2, so we can compare whether one vendor's trips are longer than the other's. `data.groupby('vendor_id')` splits the dataset into two groups based on `vendor_id`; `['trip_duration']` selects the `trip_duration` column from each group; `.mean()` calculates the average trip duration for each vendor group.

```
vendor_id
1 845.438244
2 1058.643164
Name: trip_duration, dtype: float64
```

"This tells us Vendor 2's trips are longer on average than Vendor 1's. The code grouped trips by vendor and calculated the mean trip duration for each."

Chart: "This boxplot compares trip durations between Vendor 1 and Vendor 2. Vendor 2's trips are longer on average, while Vendor 1's trips are shorter. The chart confirms the stats calculated earlier."

Step 17: Median Trip Duration, Vendor-wise

Calculating median trip duration per vendor.

```
data.groupby('vendor_id')['trip_duration'].median()

vendor_id
1 658.0
2 666.0
Name: trip_duration, dtype: float64
```

Vendor 1: typical trip duration is about 658 seconds (~11 minutes). Vendor 2: typical trip duration is about 666 seconds (~11.1 minutes). The medians are very close, meaning both vendors have similar central tendency for trip durations - but Vendor 2's average (mean) was higher because of some unusually long trips.

```
weekday_order = ['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday', 'Sunday']

data.groupby(['pickup_weekday',
              'vendor_id'])['trip_duration'].median().unstack().reindex(weekday_order).plot(kind='line',
marker='o')
plt.title('Median Trip Duration by Day of Week and Vendor')
plt.xlabel('Day of Week')
plt.ylabel('Median Trip Duration (seconds)')
plt.legend(title='Vendor ID')
plt.show()
```

"This code compares the median trip duration for Vendor 1 and Vendor 2 across weekdays. The line plot shows that trips get longer midweek, peak on Thursday, and shorten toward Sunday. Vendor 2's trips are consistently a little longer than Vendor 1's."

What the chart shows: x-axis is days of the week (Mon-Sun); y-axis is median trip duration (seconds); lines are Vendor 1 and Vendor 2. Pattern: median trip duration rises from Monday, peaks around Thursday, then drops toward Sunday. Vendor 2's line is consistently a little higher, meaning their trips are slightly longer.

Step 18: Trip Duration vs Passenger Count

Checking whether the number of passengers affects trip duration - calculating the mean trip duration for each passenger count group and plotting it.

```
data.groupby('passenger_count')['trip_duration'].mean()

passenger_count
0 1718.433333
1 930.399753
2 1005.458335
3 1028.236276
4 1053.529749
5 1070.232174
6 1061.355223
7 19.666667
8 104.000000
9 560.000000
Name: trip_duration, dtype: float64
```

A taxi can't have 0 passengers - that's a clear data error. Trips with 0 passengers have very long average durations compared to normal trips. Trips with 7, 8, or 9 passengers have strange or very short averages. These don't follow the usual pattern of passenger counts 1-6, so they are outliers.

"We say passenger count doesn't change trip duration much because trips with 1-5 passengers all last about 15-18 minutes. The difference is only a few minutes, and the boxplot shows the groups overlap a lot. So passenger count has only a weak effect compared to other factors like distance or weekday."

Step 19: Pickup Density Heatmap

Visualising the most frequently occurring pickup points on an actual latitude/longitude map, using the folium package, researched specifically for this kind of geographic visualisation.

```
import folium
from folium.plugins import HeatMap
```

folium builds interactive maps; HeatMap plots the coordinates as a density heatmap on that map instead of individual dots.

```
nyc_box = data[(data['pickup_longitude'].between(-74.05, -73.75)) &
               (data['pickup_latitude'].between(40.6, 40.9))]
```

data[...] selects rows from the dataset. pickup_longitude.between(-74.05, -73.75) keeps only pickups with longitude inside NYC's west-east range. pickup_latitude.between(40.6, 40.9) keeps only pickups with latitude inside NYC's south-north range. The & means both conditions must be true. The result (nyc_box) is a cleaned dataset containing only trips that actually happened inside NYC boundaries. "This code filters the data so you only keep taxi trips that happened inside New York City. It removes trips with coordinates far outside NYC, leaving a clean dataset of real NYC pickups."

```
sample = nyc_box.sample(5000, random_state=42)
```

"This code takes 5,000 random taxi trips from the NYC dataset so the map runs faster. The random_state=42 makes sure you always get the same 5,000 trips each time."

```
m = folium.Map(location=[40.75, -73.98], zoom_start=11)
HeatMap(data=sample[['pickup_latitude', 'pickup_longitude']].values.tolist()).add_to(m)
m
```

"This code makes an interactive NYC map. It adds a glowing heatmap layer where brighter spots show places with more taxi pickups, based on the 5,000 sampled trips."

Heatmap: "The heatmap shows where taxi pickups happen most often. Bright spots mean high demand, while lighter areas mean fewer pickups. It proves that taxi demand is concentrated in the city center, not evenly spread across NYC."

Observation Report: "The heatmap shows pickup density is heavily concentrated in Manhattan, with the brightest, most intense clustering around the island's core - matching what was already found in the coordinate distribution step. Areas outside Manhattan (outer boroughs, airports) show noticeably lighter, more scattered activity, confirming that NYC yellow taxi demand is overwhelmingly an inner-city phenomenon rather than evenly spread across the city."

Step 20: Correlation Heatmap (Raw Features)

Expected to produce a grid chart where each square shows how two variables are related. The colours make it easy to spot strong or weak relationships, and the numbers confirm the exact strength.

```
data['day_of_week'] = data['pickup_datetime'].dt.dayofweek
data['hour_of_day'] = data['pickup_hour']
```

Correlation (.corr()) only works on numeric columns - it can't calculate a correlation with text or datetime objects directly. So before building the heatmap, two clean numeric versions of time-related information were created, so they can actually be included in the correlation matrix: day_of_week turns the pickup timestamp into a number (0-6) for Mon-Sun, and hour_of_day just renames pickup_hour - both so they can be numeric inputs.

```
heatmap_cols = ['vendor_id', 'pickup_longitude', 'pickup_latitude', 'dropoff_longitude',
                'dropoff_latitude', 'store_and_fwd_flag', 'trip_duration', 'day_of_week',
                'hour_of_day']

corr_matrix = data[heatmap_cols].astype('float64').corr()

plt.figure(figsize=(9,7))
sns.heatmap(corr_matrix, cmap='YlGnBu', vmin=-1, vmax=1, square=True, annot=True, fmt='.3f')
plt.title('Correlation Heatmap - Raw Features')
plt.show()
```

"This code calculates how strongly the chosen features are related to each other and then draws a heatmap. Bright colours show strong correlations, pale colours show weak ones, and the numbers confirm the exact strength."

Report: "After fixing the colour scale to show the true range from -1 to 1, a few negative correlations that were hidden before are now visible, although none of them are very strong. The biggest negative value in the whole matrix is vendor_id vs store_and_fwd_flag at -0.080, and day_of_week vs hour_of_day is at -0.084. The main conclusion stays the same though: every raw feature shows an extremely weak correlation with trip_duration. The strongest is pickup_latitude at -0.029, barely different from pickup_longitude (0.027). The one genuinely strong relationship in the whole matrix is between pickup_longitude and dropoff_longitude at 0.784, and to a smaller extent pickup_latitude and dropoff_latitude at 0.494 - this makes sense because most trips start and end within a similar geographic area of the city. This backs up the idea that distance, which comes from combining the pickup and dropoff coordinates together, is what will actually help predict trip duration, not any single raw column on its own - which is exactly what gets engineered as a feature in the next stage (preprocessing and modelling)."

"The correlation heatmap highlights strong positive relationships between pickup and dropoff coordinates, confirming that trips generally follow consistent geographic patterns. Trip duration shows moderate correlation with distance-related features, while categorical and temporal variables (vendor ID, store-and-forward flag, day of week, hour of day) have weak correlations. This indicates that spatial variables are the most influential predictors, while vendor and time features contribute less directly."

That completes Stages 1-4: the problem is defined, hypotheses are written down, the raw data is loaded and inspected, and it has been explored in enough depth - through missing-value checks, target statistics, univariate charts, bivariate comparisons, a pickup density map, and a correlation heatmap - to move confidently into preprocessing and modelling in Stage 5.

STAGE 5 - PREDICTIVE MODELING

What was expected of me here: to take the cleaned EDA dataset and prepare it properly for machine learning - checking data quality one final time, handling outliers with justified reasoning, engineering a useful feature, splitting the data correctly, scaling it, training a KNN model and a Linear Regression model, tuning and comparing them, and diagnosing the results using residual analysis.

Step 21: Final Missing Values Check

```
data.isnull().sum()
```

"I checked missing values during EDA to understand the quality of the dataset. I checked again during preprocessing as a final confirmation that there are no missing values before the data is passed to the machine-learning model." It's basically a final safety check before modelling.

Result: No missing values are present in the dataset at the preprocessing stage. Therefore, no imputation is required.

Step 22: Duplicate Records Check

The dataset was checked for duplicate rows using `data.duplicated().sum()`. The result was 0 duplicate rows, therefore no duplicate records were removed. I checked for duplicate records because repeated observations can give some trips more weight during model training and may affect the model's results. Since there were no duplicates, no removal was necessary.

Step 23: data.dtypes - Column Review

Purpose: Check the data type of every column so I can decide how each variable should be handled before modelling.

"Drop means exclude the column from training because it's not useful or it leaks information. Keep means include it as a feature because it can help the model predict trip duration."

This is the point where I decide which variables are useful for modelling and which should be dropped or kept.

Decision rules

Target variable: trip_duration is the answer being predicted, so it becomes y (not part of X). log_trip_duration is dropped from X because it's derived directly from the target.

Leakage variables (known only after the trip): dropoff_datetime is dropped, since you only know this after the trip ends.

Identifiers: id is dropped - a random identifier with no predictive value.

Useful predictors kept: vendor_id (may capture vendor-specific differences); passenger_count (could affect trip duration); pickup_longitude, pickup_latitude, dropoff_longitude, dropoff_latitude (spatial features strongly influence trip length); store_and_fwd_flag (categorical, already encoded as int8).

Datetime features: pickup_datetime is dropped raw after extracting features from it. pickup_weekday (string) is dropped, since it duplicates day_of_week. pickup_hour is dropped, since it duplicates hour_of_day. day_of_week and hour_of_day are kept as numeric weekday/hour features.

Drop vs Keep meaning

Drop - remove the column from the modelling dataset. Reasons: it's just an identifier (id); it leaks information you wouldn't know at prediction time (dropoff_datetime); it's redundant or duplicated (pickup_weekday vs day_of_week); it's derived directly from the target (log_trip_duration).

Keep - retain the column as a predictor (feature) because it may help the model learn patterns. Examples: spatial features (pickup_longitude, dropoff_latitude), temporal features (day_of_week, hour_of_day), categorical flags (vendor_id, store_and_fwd_flag), passenger count (passenger_count).

Final presentation statement: "I used data.dtypes to identify the data types of each variable. I then decided which variables could contribute useful information to the model, which were duplicates or unsuitable, and which could cause target leakage. I kept meaningful numeric and geographic features, removed identifiers, redundant/raw variables and leakage variables, and separated trip_duration as the target variable."

Important: We have not deleted these columns from data yet at this point - this step only identifies what should happen to each column. The actual changes are made in the next step. model_data now contains 10 columns: vendor_id, passenger_count, pickup/dropoff coordinates, store_and_fwd_flag, trip_duration, day_of_week, hour_of_day. All identifier, raw-timestamp, leakage (log_trip_duration, dropoff_datetime), and duplicate (pickup_weekday, pickup_hour) columns were removed. The remaining variables are all numeric and usable directly by the models (KNN and Linear Regression).

Step 24: Outlier Handling - Trip Duration (IQR Method)

```
Q1 = model_data['trip_duration'].quantile(0.25)
Q3 = model_data['trip_duration'].quantile(0.75)
IQR = Q3 - Q1

whisker_1 = Q1 - (1.5 * IQR)
whisker_2 = Q3 + (1.5 * IQR)

whisker_1, whisker_2
```

This applies the Interquartile Range (IQR) rule for detecting outliers in the trip_duration data. Any value below whisker_1 or above whisker_2 is considered an outlier.

Output: (np.float64(-603.5), np.float64(2072.5)). Lower whisker = -603.5 - no valid trip duration will fall below this, so only the upper whisker matters. Upper whisker = 2072.5 - the maximum normal trip duration; trips longer than that are treated as outliers and can be removed.

"Those two numbers are the limits of what counts as a normal trip duration. Trips shorter than -603 seconds (impossible) or longer than 2,072 seconds are flagged as outliers." Both numbers are trip duration values expressed in seconds, but they are not actual trips - they are statistical boundaries calculated using the IQR method.

```
outliers_iqr = model_data.loc[model_data['trip_duration'] > whisker_2]
print(len(outliers_iqr))
print(len(outliers_iqr) / len(model_data) * 100)
# 74220
# 5.088287477958981
```

"This code finds all trips longer than ~34 minutes, counts how many there are, and shows what percentage of the dataset they make up." 74,220 trips are outliers by this method, about 5.09% of the dataset.

```
print("Before cleaning:", model_data.shape)

# Keep trips between 1 minute and 4 hours
model_data = model_data[(model_data['trip_duration'] >= 60) & (model_data['trip_duration'] <=
```

```
4*3600)]
```

```
print("After cleaning:", model_data.shape)
```

"This code removes trips shorter than 1 minute or longer than 4 hours. It then shows how many rows you had before and after cleaning."

Report: "Using the IQR method on trip_duration, 74,220 trips (5.09% of the data) would be flagged as outliers because they are longer than 35 minutes. I decided not to remove these trips using this method, because a taxi trip taking longer than 35 minutes is still a normal, realistic trip in a big city like New York, not necessarily an error. Instead of using the IQR cutoff, I used a simple domain based rule: keep only trips that last between 1 minute and 4 hours. This removed 10,672 trips, which is about 0.73% of the original dataset (1,458,644 rows), leaving 1,447,972 rows - much smaller and safer than the strict IQR method."

Step 25: Outlier Handling - Passenger Count (IQR Method)

```
# passenger_count outliers:
Q1 = model_data['passenger_count'].quantile(0.25)
Q3 = model_data['passenger_count'].quantile(0.75)
IQR = Q3 - Q1

whisker_1 = Q1 - (1.5 * IQR)
whisker_2 = Q3 + (1.5 * IQR)

whisker_1, whisker_2
# IQR boundaries = (np.float64(-0.5), np.float64(3.5))
```

Lower whisker = -0.5 - any passenger count less than -0.5 would be considered an outlier, but since passenger counts can't be negative, this boundary is not meaningful in practice. Upper whisker = 3.5 - means counts above 3.5 (so 4, 5, 6...) are flagged as outliers by the IQR rule.

```
# to see how many trips that would actually affect:
outliers_pass = model_data.loc[model_data['passenger_count'] > 3.5]
print(len(outliers_pass))
print(len(outliers_pass) / len(model_data) * 100)
# 153849
# 10.625136397665148
```

This code tells you how many trips had 4 or more passengers: 153,849 trips (~10.63% of the dataset) have a passenger_count greater than 3.5. But in reality, NYC taxis can carry up to 6 passengers, so 4-6 are valid trips.

```
print("Before cleaning:", model_data.shape)

model_data = model_data[(model_data['passenger_count'] >= 1) & (model_data['passenger_count']
<= 6)]

print("After cleaning:", model_data.shape)
# Before cleaning: (1447972, 10) After cleaning: (1447953, 10)
```

This removes trips with 0 passengers or more than 6 passengers, then shows how many rows there were before and after cleaning.

Report: "Using the IQR method on passenger_count, the upper whisker came out to 3.5, and the lower whisker was -0.5, not meaningful since a trip cannot have negative passengers. 153,849 trips (10.63%) would be flagged as outliers because they have 4 or more passengers - but carrying 4, 5, or 6 passengers is a normal, realistic situation for a taxi. Instead of the IQR cutoff, I used a domain based rule: keep only trips with between 1 and 6 passengers. This removed 19 trips out of 1,447,972, about 0.001% of the data, leaving 1,447,953 rows."

Step 26: Outlier Handling - Coordinates (Pickup/Dropoff)

```
Q1 = model_data['pickup_longitude'].quantile(0.25)
Q3 = model_data['pickup_longitude'].quantile(0.75)
IQR = Q3 - Q1
whisker_1 = Q1 - (1.5 * IQR)
whisker_2 = Q3 + (1.5 * IQR)
whisker_1, whisker_2
# (np.float64(-74.02855300903323), np.float64(-73.93074417114255))
```

"These whisker values mean normal pickups are between -74.03 and -73.93 longitude. Any pickup outside that range is flagged as an outlier."

```
# Checking the summary stats (min, max, etc.) for all four coordinate columns, to summarize the
# distribution and quickly spot potential outliers (confirming the dataset is mostly NYC trips)
model_data[['pickup_longitude', 'pickup_latitude', 'dropoff_longitude',
'dropoff_latitude']].describe()
```

Evidence of outliers: normal NYC range is roughly longitude -74.05 to -73.75, latitude 40.5 to 41. My min/max values: longitude min = -121.93, max = -61.33, way outside NYC (California and Florida ranges); latitude min = 32.18, max = 51.88, also far outside NYC (Texas and Canada ranges).

```
# Checking how many trips fall outside a realistic NYC boundary box
outside_nyc = model_data.loc[
~model_data['pickup_longitude'].between(-74.05, -73.75) |
~model_data['pickup_latitude'].between(40.60, 40.90) |
~model_data['dropoff_longitude'].between(-74.05, -73.75) |
~model_data['dropoff_latitude'].between(40.60, 40.90)
]
print(len(outside_nyc))
print(len(outside_nyc) / len(model_data) * 100)
# 6915
# 0.47757074987931236
```

"6,915 trips (approximately 0.48%) are outside NYC's valid map range." These have to be removed - they are impossible trips (coordinates in California, Texas, Canada).

```
print("Before cleaning:", model_data.shape)
model_data = model_data[
model_data['pickup_longitude'].between(-74.05, -73.75) &
model_data['pickup_latitude'].between(40.60, 40.90) &
model_data['dropoff_longitude'].between(-74.05, -73.75) &
model_data['dropoff_latitude'].between(40.60, 40.90)
]
print("After cleaning:", model_data.shape)
# Before cleaning: (1447953, 10) After cleaning: (1441038, 10)
```

Report: "Using a realistic NYC boundary box (longitude between -74.05 and -73.75, latitude between 40.60 and 40.90), only 6,915 trips (0.48%) fell outside this range - a much smaller and safer amount of data to remove compared to the strict IQR method, which would have flagged 82,350 trips. This confirms that most of the coordinate outliers were genuine GPS errors far outside the city, not normal edge-of-city trips."

Summary of total data lost through outlier handling: started at 1,458,644 rows; after trip_duration cleaning: 1,447,972; after passenger_count cleaning: 1,447,953; after coordinate cleaning: 1,441,038 rows (final). Total removed: 17,606 rows, about 1.2% of the original dataset.

Step 27: Feature Engineering - Trip Distance

Feature engineering is the process of creating, transforming, or combining raw data into new variables (features) that make patterns easier for analysis or a machine learning model to detect.

"I added this step because coordinates alone aren't useful. Turning them into trip distance makes the data easier to understand and gives the model a stronger feature for predicting trip time and fare." Distance consistently shows strong correlation with trip duration and fare (price). Since distance is strongly linked to trip duration, I created and calculated the trip_distance_km using the Haversine formula, to help predict trip duration.

```
def haversine(lat1, lon1, lat2, lon2):
    R = 6371 # Earth's radius in km

    lat1, lon1, lat2, lon2 = map(np.radians, [lat1, lon1, lat2, lon2])

    dlat = lat2 - lat1
    dlon = lon2 - lon1

    a = np.sin(dlat/2)**2 + np.cos(lat1) * np.cos(lat2) * np.sin(dlon/2)**2
    c = 2 * np.arcsin(np.sqrt(a))

    return R * c

model_data['trip_distance_km'] = haversine(
    model_data['pickup_latitude'],
    model_data['pickup_longitude'],
    model_data['dropoff_latitude'],
    model_data['dropoff_longitude']
)

model_data['trip_distance_km'].describe()
```

Code explanation

Function definition: def haversine(lat1, lon1, lat2, lon2): defines a function that takes four inputs - the latitude and longitude of pickup (lat1, lon1) and dropoff (lat2, lon2). R = 6371 sets Earth's radius in kilometres, needed for the formula.

Convert degrees to radians: GPS coordinates are in degrees, but trigonometric functions (sin, cos) require radians, so this line converts all four values into radians.

Differences in coordinates: dlat and dlon calculate the difference between pickup and dropoff latitude/longitude. These differences are used in the Haversine formula.

Haversine formula: a is the core of the equation, combining latitude and longitude differences; c is the central angle between the two points on Earth's sphere.

Final distance: returns R * c, multiplying Earth's radius by the central angle to get the great-circle distance in kilometres.

Apply to dataset: calls the haversine function for every row in model_data, creating a new column trip_distance_km with the calculated distance for each trip.

Summarize results: model_data['trip_distance_km'].describe() generates summary statistics (count, mean, min, max, quartiles) for the new distance column, to check if values look reasonable (e.g. no negative distances, no trips hundreds of km long).

"This code defines a formula to calculate trip distance from GPS coordinates, applies it to every trip in the dataset, stores the result in a new column, and then summarizes the distances to check if they make sense."

```
count 1.441038e+06
mean 3.387194e+00
std 3.777935e+00
min 0.000000e+00
25% 1.241645e+00
50% 2.097103e+00
75% 3.851502e+00
max 3.308128e+01
Name: trip_distance_km, dtype: float64
```

Summary stats explained: count = 1,441,038 trips after cleaning. mean = 3.39 km - on average, trips are about 3.4 km long. std = 3.78 km - most trips are within +/-3.8 km of the mean. min = 0.00 km - some trips have zero distance (likely GPS errors or missing data). 25% = 1.24 km - a quarter of trips are shorter than 1.24 km. 50% (median) = 2.10 km - half of trips are shorter than ~2.1 km. 75% = 3.85 km - three quarters of trips are shorter than ~3.9 km. max = 33.08 km - the longest trip recorded, still realistic within NYC (e.g. JFK to Staten Island).

A taxi ride can't have zero distance, so checking and removing them where justified improves data quality.

```
# Checking how many trips have a distance of exactly 0 km (pickup and dropoff at the same
point)
zero_distance = model_data.loc[model_data['trip_distance_km'] == 0]
print(len(zero_distance))
print(len(zero_distance) / len(model_data) * 100)
# 4027
# 0.2794513399369066
```

4,027 trips (approximately 0.28%) have exactly 0 km distance. Since this is only 0.28% of the dataset, they don't distort the overall distribution.

Report: "There are 4,027 trips, representing about 0.28% of the dataset, where the calculated distance is 0 km. These observations were not automatically removed because a zero straight-line distance does not necessarily mean the trip is invalid - the pickup and dropoff coordinates may have been recorded at the same or rounded location, even though the taxi may have travelled a short distance. Since these represent only a very small percentage of the dataset, they were retained."

Step 28: Separate Features (X) and Target (y)

Separating features (X) and target (y) now that outlier cleaning and feature engineering are complete. We separate X and y because the model needs inputs (features) and an output (target). If we don't separate them, the model could cheat or fail to learn correctly.

```
X = model_data.drop(columns=['trip_duration'])
y = model_data['trip_duration']

print(X.shape, y.shape)
print(X.columns.tolist())

# (1441038, 10) (1441038,)
# ['vendor_id', 'passenger_count', 'pickup_longitude', 'pickup_latitude',
# 'dropoff_longitude', 'dropoff_latitude', 'store_and_fwd_flag',
# 'day_of_week', 'hour_of_day', 'trip_distance_km']
```

"We now have 10 input features in X and trip_duration as y. This setup ensures the model learns to predict duration from vendor, passenger count, coordinates, time features, and trip distance."

Step 29: Train/Test Split

Splitting the dataset so the model learns on 80% of the trips and is tested on the remaining 20%. This way, we can check if it predicts trip duration correctly on new data.

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print("Train:", X_train.shape)
print("Test:", X_test.shape)

# Train: (1152830, 10)
# Test: (288208, 10)
```

Interpretation: Train (1,152,830, 10) - the training set has 1,152,830 trips with 10 features each; this is the data the model learns patterns from. Test (288,208, 10) - the testing set has 288,208 trips with the same 10 features; this is unseen data used to evaluate how well the model generalizes. Ratio check: 80% training, 20% testing, exactly as set with test_size=0.2.

"We trained the model on about 1.15 million trips and reserved 288k trips for testing. This ensures the model learns patterns from most of the data but is evaluated on new data to check accuracy."

Step 30: Feature Scaling

"This code makes sure all features are on the same scale. Without scaling, big-number features like coordinates would overpower small ones like passenger count. Scaling balances them so the model treats each feature fairly."

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
print("Scaling done")
```

"Scaling done" - "That block of code balances all the features so the model treats them fairly. The message 'Scaling done' just tells me the process worked."

Report: All features were scaled using StandardScaler, which transforms each feature to have a mean of 0 and a standard deviation of 1. This is especially important for KNN, since it is a distance-based algorithm - if features are left on different scales, features with larger numeric ranges (like the coordinates) would dominate the distance calculation and unfairly outweigh smaller-scale features (like passenger_count). The scaler was fit only on the training data and then used to transform both the training and test sets, to avoid data leakage.

Step 31: Baseline KNN Model (K=5)

What is a base model (baseline model)

Definition: a very simple model used as a starting point. Purpose: to set a benchmark - if a more advanced model cannot beat the base model, it is not useful.

"I first built a base KNN regression model using K = 5. I then evaluated different K values to determine which value produced the best performance. Based on the comparison, K = 15 was selected as the final KNN model."

```

from sklearn.neighbors import KNeighborsRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np

knn = KNeighborsRegressor(n_neighbors=5)
knn.fit(X_train_scaled, y_train)

y_pred = knn.predict(X_test_scaled)

mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
r2 = r2_score(y_test, y_pred)

print(f"MAE: {mae:.2f} seconds")
print(f"RMSE: {rmse:.2f} seconds")
print(f"R2: {r2:.4f}")

```

Step by step explanation

knn = KNeighborsRegressor(n_neighbors=5) creates a KNN regression model with 5 neighbors; knn.fit(X_train_scaled, y_train) trains it on the scaled training data to learn how features relate to trip duration. y_pred = knn.predict(X_test_scaled) uses the trained model to predict trip durations for the test set. mae, rmse, and r2 calculate three evaluation metrics: MAE is the average error in seconds (absolute difference); RMSE is the root mean squared error, which penalizes big mistakes more; R2 shows how well the model explains variation in trip duration (closer to 1 is better).

Best practice angle: MAE tells you the average error in seconds. RMSE shows how big errors affect the model. R2 shows how much of the variation in trip duration is explained by the features.

```

MAE: 208.45 seconds
RMSE: 335.93 seconds
R2: 0.7337

```

MAE: 208.45 seconds - on average, the model's predictions are about 208 seconds (~3.5 minutes) off from the actual trip duration. RMSE: 335.93 seconds - the typical error is around 336 seconds (~5.6 minutes); RMSE penalizes larger mistakes more than MAE, so this shows some trips had bigger errors. R2: 0.7337 - the model explains about 73% of the variation in trip duration, a solid score meaning the features used (vendor, passenger count, coordinates, time, distance) capture most of the patterns in the data.

Step 32: Choosing the Best K

"This code checks how the choice of K in KNN affects the model. Small K values make the model memorize (overfit), large K values make it too simple (underfit), and somewhere in the middle gives the best balance. The plot shows train vs test R2 so I can see where the model works best."

```

from sklearn.neighbors import KNeighborsRegressor
from sklearn.metrics import r2_score
import matplotlib.pyplot as plt

train_data = X_train.copy()
train_data['trip_duration'] = y_train.values

sample = train_data.sample(n=50000, random_state=42)
X_sample = sample.drop(columns=['trip_duration'])
y_sample = sample['trip_duration']

Xs_train, Xs_test, ys_train, ys_test = train_test_split(
    X_sample, y_sample, test_size=0.2, random_state=42
)

```

```

)

sample_scaler = StandardScaler()
Xs_train_scaled = sample_scaler.fit_transform(Xs_train)
Xs_test_scaled = sample_scaler.transform(Xs_test)

k_values = [1, 3, 5, 9, 15, 25, 50, 75, 100]
train_scores = []
test_scores = []

for k in k_values:
    model = KNeighborsRegressor(n_neighbors=k, n_jobs=-1)
    model.fit(Xs_train_scaled, ys_train)
    train_scores.append(r2_score(ys_train, model.predict(Xs_train_scaled)))
    test_scores.append(r2_score(ys_test, model.predict(Xs_test_scaled)))

plt.figure(figsize=(8,5))
plt.plot(k_values, train_scores, marker='o', color='green', label='Train R2')
plt.plot(k_values, test_scores, marker='o', color='red', label='Test R2')
plt.xlabel('K (number of neighbors)')
plt.ylabel('R2 Score')
plt.title('Underfitting vs Overfitting: Train vs Test R2 by K')
plt.legend()
plt.grid(True)
plt.show()

print(list(zip(k_values, train_scores, test_scores)))

```

Understanding the plot

Small K (like 1, 3, 5): Train R2 is very high (close to 1.0), the model memorizes training data; Test R2 is much lower, poor generalization - this is overfitting. Large K (like 75, 100): Train R2 drops, the model becomes too simple; Test R2 also flattens around ~0.7, the model cannot capture enough detail - this is underfitting. Middle K (like 9-25): Train R2 is lower than 1 but still strong; Test R2 is higher and closer to train R2 - this balance means the model generalizes better.

"When K is small, the model memorizes the training data and fails on new data (overfitting). When K is very large, the model becomes too simple and misses patterns (underfitting). In the middle range, the train and test scores are balanced, which is the sweet spot."

Best practice angle: the plot helps choose the right K - you want the point where test R2 is highest and not too far below train R2. In this case, that looks to be around K = 15-25, where test R2 is ~0.7 and train R2 is not too high.

```

[(1, 1.0, 0.46407986488621644), (3, 0.8261897784363741, 0.6493726260905204),
(5, 0.7865919069751605, 0.6832928997831477), (9, 0.7547155810544497, 0.6998934790420035),
(15, 0.7338345123016741, 0.6989580415845558), (25, 0.7137359486253951, 0.6939812079909033),
(50, 0.6872980580404237, 0.6799179239577992), (75, 0.6712059791213785, 0.6675814790112906),
(100, 0.6590810637321454, 0.657551206569738)]

```

KNN fit status: K = 1, Train R2 = 1.0, Test R2 ~ 0.46 - the model memorizes training data but fails on test data, this is overfitting. K = 3-5, Train R2 drops slightly, Test R2 improves (~0.65-0.68) - still leaning toward overfitting, but better than K=1. K = 9-25, Train R2 ~ 0.71-0.75, Test R2 ~ 0.69-0.70 - train and test scores are balanced, this is the best-fit region. K = 50-100, Train R2 ~ 0.66-0.69, Test R2 ~ 0.66-0.68 - both scores are lower, meaning the model is too simple, this is underfitting.

"At K=1 the model is overfitting. At very large K (like 75 or 100) the model is underfitting. In the middle range (around 15-25), the model is best-fit because train and test scores are balanced and test R2 is highest."

Report: To find a good value of K without testing every K on the full 1.4 million row dataset, a random sample of 50,000 training rows was taken and K = 1, 3, 5, 9, 15, 25, 50, 75, and 100 were tested. K=9 and K=15 gave very similar test performance (0.6999 and 0.6990), both clearly better than very low or very high K. Since K=9 and K=15 were so close, 5-fold cross-validation was used next to make a more reliable choice.

Step 33: Cross-Validation to Confirm K

"Testing KNN with cross-validation. The mean R2 shows how well the model works overall, and the std shows how consistent it is. A good K has a high mean R2 and a low std."

```
from sklearn.model_selection import cross_val_score
from sklearn.pipeline import Pipeline

cv_means = []
cv_stds = []

for k in k_values:
    pipe = Pipeline([
        ('scaler', StandardScaler()),
        ('knn', KNeighborsRegressor(n_neighbors=k, n_jobs=-1))
    ])
    scores = cross_val_score(pipe, X_sample, y_sample, cv=5, scoring='r2')
    cv_means.append(scores.mean())
    cv_stds.append(scores.std())

for k, m, s in zip(k_values, cv_means, cv_stds):
    print(f"K={k}: Mean R2={m:.4f}, Std={s:.4f}")

K=1: Mean R2=0.4833, Std=0.0136
K=3: Mean R2=0.6467, Std=0.0077
K=5: Mean R2=0.6761, Std=0.0079
K=9: Mean R2=0.6924, Std=0.0085
K=15: Mean R2=0.6952, Std=0.0092
K=25: Mean R2=0.6901, Std=0.0094
K=50: Mean R2=0.6754, Std=0.0097
K=75: Mean R2=0.6628, Std=0.0096
K=100: Mean R2=0.6527, Std=0.0094
```

K=1: Mean R2 0.4833, Std 0.0136 - overfitting (memorizes training, poor generalization). K=3: 0.6467, 0.0077 - improving, still leaning overfit. K=5: 0.6761, 0.0079 - stronger, more stable. K=9: 0.6924, 0.0085 - balanced, close to best fit. K=15: 0.6952, 0.0092 - best fit (highest mean R2, low std). K=25: 0.6901, 0.0094 - still good, slightly lower. K=50: 0.6754, 0.0097 - declining, underfitting begins. K=75: 0.6628, 0.0096 - clear underfitting. K=100: 0.6527, 0.0094 - strong underfitting.

"At K=1 the model is overfitting. At very large K (75-100) the model is underfitting. The best performance is around K=15, where the mean R2 is highest (~0.70) and the scores are stable across folds."

Report: To confirm the best K more reliably than a single train/test split, 5-fold cross-validation was used, which splits the sample into 5 parts, trains and tests the model 5 times, and averages the results. K=15 had the highest mean R2 (0.6952), slightly ahead of K=9 (0.6924), confirming K=15 as the best choice. Since cross-validation is specifically meant to give the most reliable answer, and K=15 had the highest score, K=15 was chosen to stay consistent with the method being used.

Step 34: Final KNN Model (K=15)

The purpose of the final KNN model section is to show the final, optimized version of the model after all the testing, tuning, and validation steps - trained on the full training set using the best K found by cross-validation (K=15).

```
final_knn = KNeighborsRegressor(n_neighbors=15, n_jobs=-1)
final_knn.fit(X_train_scaled, y_train)

final_pred_knn = final_knn.predict(X_test_scaled)

final_mae_knn = mean_absolute_error(y_test, final_pred_knn)
final_rmse_knn = np.sqrt(mean_squared_error(y_test, final_pred_knn))
final_r2_knn = r2_score(y_test, final_pred_knn)

print(f"FINAL KNN MODEL (K=15)")
print(f"MAE: {final_mae_knn:.2f} seconds")
print(f"RMSE: {final_rmse_knn:.2f} seconds")
print(f"R2: {final_r2_knn:.4f}")

FINAL KNN MODEL (K=15)
MAE: 199.90 seconds
RMSE: 322.82 seconds
R2: 0.7540
```

After tuning and confirming the best K through cross-validation, the final KNN model was trained on the full dataset using K=15 neighbors. The evaluation on the test set produced: MAE 199.90 seconds - on average, predictions are about 3 minutes 20 seconds off from the actual trip duration. RMSE 322.82 seconds - typical error is around 5 minutes 23 seconds, with larger mistakes penalized more. R2 0.7540 - the model explains about 75% of the variation in trip duration, which is strong performance and confirms good generalization.

Step 35: Linear Regression Model

Training a Linear Regression model for comparison against KNN (baseline model).

```
from sklearn.linear_model import LinearRegression

lr = LinearRegression()
lr.fit(X_train_scaled, y_train)

lr_pred = lr.predict(X_test_scaled)

lr_mae = mean_absolute_error(y_test, lr_pred)
lr_rmse = np.sqrt(mean_squared_error(y_test, lr_pred))
lr_r2 = r2_score(y_test, lr_pred)

print(f"LINEAR REGRESSION")
print(f"MAE: {lr_mae:.2f} seconds")
print(f"RMSE: {lr_rmse:.2f} seconds")
print(f"R2: {lr_r2:.4f}")
```

Evaluation: MAE shows the average error in seconds - if higher than KNN's 199.90 seconds, KNN is more accurate. RMSE penalizes larger errors - compare to KNN's 322.82 seconds. R2 shows how much variation in trip duration is explained - compare to KNN's 0.7540.

"Linear Regression gives me a baseline performance. By comparing its MAE, RMSE, and R2 to the tuned KNN model, I can see whether KNN actually improves prediction accuracy."

```
LINEAR REGRESSION
MAE: 277.96 seconds
RMSE: 412.05 seconds
R2: 0.5993
```

Results: MAE 277.96 seconds - average error is about 4 minutes 38 seconds. RMSE 412.05 seconds - typical error is about 6 minutes 52 seconds, with larger mistakes penalized more. R2 0.5993 - explains about 60% of the variation in trip duration.

"Linear Regression predicts trip duration with an average error of nearly 5 minutes and explains about 60% of the variation. This is weaker than the tuned KNN model, which had an average error of 3 minutes and explained 75% of the variation."

Step 36: Model Comparison Table

```
# Compare both models side by side
comparison_models = pd.DataFrame({
    'Model': ['KNN (K=15)', 'Linear Regression'],
    'MAE': [final_mae_knn, lr_mae],
    'RMSE': [final_rmse_knn, lr_rmse],
    'R2': [final_r2_knn, lr_r2]
})
comparison_models

Model MAE RMSE R2 Fit Status
KNN (K=15) 199.90 322.82 0.7540 Best Fit
Linear Regression 277.96 412.05 0.5993 Baseline, weaker
```

"KNN with K=15 predicts trip duration more accurately than Linear Regression. It reduces the average error by about 80 seconds and explains 15% more of the variation in trip times."

Conclusion for report: This comparison confirms that KNN (K=15) is the superior model. It consistently outperforms Linear Regression across all metrics (MAE, RMSE, R2), proving that the tuning and cross-validation process led to a stronger, more generalizable model.

Step 37: Linear Regression Coefficients

Model interpretation step: inspecting `lr.coef_` (and `lr.intercept_`) to understand how each feature influences predictions. "`lr.coef_` is part of the interpretation step. After training and evaluating the model, you look at the coefficients to see which features push predictions up or down."

```
# Show the linear regression coefficients: this tells us how much each feature
# affects the predicted trip_duration (positive = increases duration, negative = decreases it)
lr.coef_

array([-4.42888051e-02,  3.96736443e+00, -2.83091393e+01,  3.43153482e+01,
        -4.77781910e+01, -3.91129323e+01,  3.25341140e+00, -2.58274417e+01,
         2.46831611e+01,  5.39159496e+02])
```

This array represents the coefficients (weights) of the Linear Regression model for each feature in the dataset. "These numbers show how each input feature affects the predicted trip duration. Positive values increase the prediction, negative values decrease it. Because the data was scaled, the magnitudes are directly comparable."

How to use them: positive coefficients (e.g. +539.16) are features that strongly increase predicted trip duration; negative coefficients (e.g. -47.78, -39.11) are features that reduce predicted trip duration; small coefficients (e.g. -0.044) have little influence. Since `StandardScaler` was used, magnitudes can be compared to see which features matter most.

```
# Pair each coefficient with its feature name so the numbers are interpretable
coef_df = pd.DataFrame({
    'Feature': X.columns,
    'Coefficient': lr.coef_
})
coef_df
```

```
Feature Coefficient
0 vendor_id -0.044289
1 passenger_count 3.967364
2 pickup_longitude -28.309139
3 pickup_latitude 34.315348
4 dropoff_longitude -47.778191
5 dropoff_latitude -39.112932
6 store_and_fwd_flag 3.253411
7 day_of_week -25.827442
8 hour_of_day 24.683161
9 trip_distance_km 539.159496
```

Interpretation by feature: vendor_id (-0.044) - negligible effect, essentially irrelevant. passenger_count (+3.97) - slight positive influence, more passengers slightly increase predicted trip duration. pickup_longitude (-28.31) - moderate negative influence, certain pickup locations reduce predicted duration. pickup_latitude (+34.32) - moderate positive influence, certain pickup locations increase predicted duration. dropoff_longitude (-47.78) - strong negative influence. dropoff_latitude (-39.11) - strong negative influence. store_and_fwd_flag (+3.25) - very small positive effect, minimal importance. day_of_week (-25.83) - moderate negative influence, trips on certain days are shorter. hour_of_day (+24.68) - moderate positive influence, trips at certain hours are longer. trip_distance_km (+539.16) - dominant driver, longer distances massively increase predicted trip duration.

"The model uses vendor ID, passenger count, pickup/drop-off coordinates, store-and-forward flag, day of week, hour of day, and trip distance. Of these, trip distance is by far the strongest predictor of trip duration, while time of day and location also matter. Vendor ID and store-and-forward flag barely affect predictions."

Report: Since the features were scaled before training, the size of each coefficient reflects how strongly that feature influences the prediction. trip_distance_km has by far the largest coefficient at about 539 - a one-standard-deviation increase in trip distance is associated with roughly a 539 second increase in predicted trip duration, holding other scaled features constant (it does not mean "1 km adds 539 seconds"). This matches the original hypothesis that longer distances take longer. vendor_id has almost no effect (-0.044), suggesting which company operated the trip barely matters for predicting duration once the other features are included.

Step 38: Coefficient Plot

```
plt.figure(figsize=(10, 6))
plt.bar(coef_df['Feature'], coef_df['Coefficient'])
plt.xticks(rotation=45, ha='right')
plt.ylabel('Coefficient value')
plt.title('Linear Regression Coefficients')
plt.axhline(y=0, color='red', linewidth=0.8)
plt.tight_layout()
plt.show()
```

What the chart shows: trip_distance_km dominates completely - its bar reaches about 540, dwarfing every other feature by roughly 10-15x. This visually confirms it's by far the strongest predictor in the model, matching what was already seen in the correlation heatmap discussion (raw features barely correlated with trip_duration individually, but the engineered distance feature clearly does). vendor_id, passenger_count, and store_and_fwd_flag sit essentially flat at zero - their bars are barely visible, meaning almost no linear influence on the predicted duration. pickup_longitude (~-34), dropoff_longitude (~-48), dropoff_latitude (~-39), and

day_of_week (~26) all show small-to-moderate bars, some positive and some negative, with some effect but nowhere near trip_distance_km. hour_of_day has a small positive bar (~25), meaning trips later in the day are predicted to be slightly longer, all else being equal.

Report: The bar chart makes the pattern from the coefficient table easier to see at a glance. The trip_distance_km bar is far taller than all the others, visually confirming it has the strongest contribution to the prediction. Most other features sit close to zero, with only the coordinates, hour_of_day, and day_of_week showing a moderate effect in either direction, while vendor_id and store_and_fwd_flag are barely visible on the chart.

Step 39: Residuals Table and Residual Plot

```
residuals = pd.DataFrame({'fitted values': y_test, 'predicted values': lr_pred})
residuals['residuals'] = residuals['fitted values'] - residuals['predicted values']

plt.scatter(residuals['predicted values'], residuals['residuals'], alpha=0.3, s=5)
plt.axhline(y=0, color='red')
plt.title('Residual Plot')
plt.show()
```

Residuals were calculated as the difference between the actual trip duration and the predicted trip duration for each trip in the test set. A positive residual means the model underestimated the trip duration; a negative residual means it overestimated it.

Report: The residual plot shows the errors are not evenly scattered around zero. Most residuals sit above the red zero line, especially at lower predicted values, meaning the model tends to underestimate trip duration for shorter trips. The spread of residuals is also wider on the left side and narrower on the right, forming a funnel-like shape - a sign the linear regression model doesn't fit the data equally well across all trip durations, likely because the relationship between the features and trip duration isn't fully linear.

Step 40: Histogram of Residuals

```
plt.hist(residuals['residuals'], bins=150)
plt.title('Distribution of Residuals')
plt.show()
```

Report: The histogram of residuals shows a shape that is roughly bell-like and centered close to zero, meaning most predictions are fairly close to the actual trip duration. However, the distribution is not perfectly symmetrical - there is a longer tail stretching to the right, meaning a number of trips were significantly underestimated by the model. This matches what was seen in the residual plot.

Step 41: QQ Plot of Residuals

```
from statsmodels.graphics.gofplots import qqplot
fig, ax = plt.subplots(figsize=(6,6), dpi=100)
qqplot(residuals['residuals'], line='s', ax=ax)
plt.title('QQ Plot of Residuals')
plt.show()
```

Report: The QQ plot compares the residuals to what a perfectly normal distribution would look like. In the middle section, the blue points follow the red line reasonably closely, meaning most residuals are close to normally distributed. However, the points curve sharply upward and away from the red line at the higher end, meaning there are more extreme positive residuals than a normal distribution would predict - confirming the long tail seen in the histogram, and a limitation of the linear regression model on this dataset.

Step 42: Overall Summary of the Diagnostics

The Linear Regression diagnostic plots show that the model does not fully capture the patterns in trip duration. The residual plot shows errors spread around the zero line, with larger errors appearing for some longer trips. The residual histogram and QQ plot also show that the errors are not perfectly normally distributed. Linear Regression achieved an R^2 of 0.599, while KNN achieved an R^2 of 0.754, with lower MAE and RMSE also on KNN's side. This suggests KNN captured the relationships in this dataset better than Linear Regression. Therefore, based on the evaluation results, KNN performed better than Linear Regression for predicting taxi trip duration in this project.

Step 43: Final Conclusion

"The goal of this project was to predict NYC taxi trip duration using the trip and location data provided. From the EDA, it was clear trip duration is affected by several things - time of day and day of the week both showed patterns (e.g. certain hours had longer average trips), and vendor and passenger count showed some observable differences too, although weaker. However, once the Linear Regression model was built, the coefficients showed passenger_count still had a small positive effect, but vendor_id contributed almost nothing (-0.044) once the other features were included, suggesting the vendor differences seen in EDA don't hold up as a meaningful predictor in the final model."

"For preprocessing, I checked for missing values and duplicates and found none. I removed columns that were either irrelevant (id, raw timestamps) or leakage (log_trip_duration, since it's directly derived from the target). For outliers in trip_duration, passenger_count, and the coordinates, I didn't just use the standard IQR method, because it would have removed a large amount of valid data (over 74,000 trips in the case of trip_duration alone). Instead I used domain based cutoffs which removed a much smaller, more justified portion of the data overall. I also engineered a new feature, trip_distance_km, calculated from the pickup and dropoff coordinates, since raw latitude/longitude values don't represent actual distance on their own."

"I trained and compared two models: KNN Regression and Linear Regression. For KNN, I tested several values of K and used 5-fold cross-validation to confirm the best one, which came out to $K=15$. The final results were: KNN ($K=15$): MAE = 199.90 seconds, RMSE = 322.82 seconds, $R^2 = 0.754$; Linear Regression: MAE = 277.96 seconds, RMSE = 412.05 seconds, $R^2 = 0.599$. KNN performed noticeably better across all three metrics. This is also supported by the Linear Regression diagnostics - the residual plot showed a funnel shape instead of scattering randomly around zero, and the QQ plot showed the residuals moving away from normal at the higher end. This suggests the relationship between the features and trip duration isn't fully linear, which is likely why Linear Regression underperformed compared to KNN."

"Based on these results, KNN ($K=15$) is the better model for this dataset."